Escuela de Ciencia y Tecnología
ECyT_UNSAM

Ingeniería de la Rehabilitación

Trabajo Práctico Etapa Nº1

Investigación y Desarrollo del Módulo de Evaluación

Exploración de Faro (Neglect)

Profesor: Crespo, Marcos

1er cuatrimestre 2026

GRUPO 2

Camila Isabel Dorado

Lara Gotthelf

Elena Passeri

Priscila Rettura

1. Investigación clínica y búsqueda de evidencia **2**

1.1 Introducción 2

1.2 Estado del arte 4

1.3 Definición de métricas: valores “normales” y “patológicos” 7

2. Desarrollo de la herramienta **9**

3. Análisis de un caso con desempeño dentro de valores normales **10**

3.1 Evaluación de métricas y comparación con bibliografía 12

Anexo I: Código final **14**

Anexo II: Prompts utilizados con la IA **34**

1. Investigación clínica y búsqueda de evidencia

1.1 Introducción

El accidente cerebrovascular (ACV), ya sea de origen isquémico o hemorrágico, constituye una condición altamente discapacitante que puede comprometer múltiples dominios funcionales, incluyendo la cognición, la función visuoespacial, el lenguaje y la motricidad. En particular, cuando el ACV afecta al lóbulo parietal, se altera la capacidad del cerebro para integrar e interpretar la información sensorial, lo que repercute directamente en la percepción espacial y la representación del entorno y del propio cuerpo [1].

Desde el punto de vista neurofuncional, el lóbulo parietal del hemisferio derecho desempeña un rol predominante en los procesos de atención espacial y conciencia del entorno. Por este motivo, las lesiones en esta región suelen generar déficits más marcados que las del hemisferio izquierdo, especialmente en relación con la percepción del espacio contralateral (habitualmente el lado izquierdo) [2].

Entre las principales alteraciones asociadas a un ACV en el lóbulo parietal derecho se encuentran [3]:

- Heminegligencia (negligencia espacial unilateral): trastorno atencional caracterizado por una disminución o ausencia de respuesta a estímulos presentados en el lado contralateral a la lesión cerebral, en ausencia de un déficit sensorial primario. Clínicamente, suele manifestarse como negligencia izquierda, donde el paciente presenta escasa conciencia del lado izquierdo de su cuerpo o del espacio circundante. Es importante destacar que no se trata de un problema visual, sino de una alteración en los mecanismos de atención y procesamiento espacial.

[1] Visual Neglect Following Stroke: Current Concepts and Future Focus - ScienceDirect
[2] Spatial neglect and attention networks - PubMed
[3] Ictus del lóbulo parietal: qué esperar y cómo recuperarse

- Alteración de la percepción espacial: dificultad para comprender la posición del cuerpo en relación con el entorno, lo que afecta la orientación y la interacción con objetos. Esta alteración compromete la capacidad del individuo para desenvolverse adecuadamente en su entorno.
- Trastornos de la orientación y memoria visual: incluyen dificultades para recordar configuraciones espaciales o reconocer entornos, lo que impacta negativamente en la autonomía funcional y en la ejecución de actividades de la vida diaria.
- Anosognosia: incapacidad para reconocer los propios déficits neurológicos. Esta condición tiene implicancias clínicas relevantes, ya que el paciente puede negar o minimizar sus limitaciones, reduciendo su adherencia a los procesos de rehabilitación.
- Disfunción propioceptiva: alteración en la percepción de la posición y movimiento de las partes del cuerpo, lo que contribuye a problemas de equilibrio y coordinación motora. Si bien puede observarse en lesiones de ambos hemisferios, es más frecuente en lesiones del hemisferio derecho.

En este contexto, la heminegligencia espacial unilateral representa uno de los síndromes más relevantes desde el punto de vista clínico y de la rehabilitación. Se define como un trastorno neurológico en el cual el paciente no percibe ni responde a estímulos en un lado del espacio o del cuerpo, generalmente contralateral a la lesión. Este síndrome es particularmente frecuente tras lesiones del hemisferio derecho y se caracteriza por la coexistencia de déficits espaciales y no espaciales.

En términos funcionales, estos pacientes pueden comportarse como si el hemiespacio contralateral no existiera, lo que se evidencia en situaciones cotidianas; por ejemplo, pueden no responder a estímulos ubicados en ese lado o ignorar completamente objetos o personas presentes en dicha región del espacio. Este cuadro no debe confundirse con defectos del campo visual (como la hemianopsia), ya que en la

heminegligencia el problema radica en la atención y representación espacial, y no en la recepción sensorial visual [4].

1.2 Estado del arte

En la práctica clínica, la evaluación del *neglect* espacial unilateral sigue apoyándose principalmente en pruebas convencionales de papel y lápiz, especialmente tareas de cancelación, bisección de líneas, copiado, lectura/escritura y pruebas conductuales o funcionales. Una revisión sistemática con recomendaciones de la *European Academy of Neurology* analizó 42 estudios y concluyó que las pruebas pueden agruparse precisamente en esas familias. Además, informó que las tareas de cancelación fueron las más utilizadas en la literatura (97,6% de los estudios incluidos) y recomendó una prueba de cancelación como *screening* inicial cuando solo puede administrarse un test. La misma revisión remarca que ningún test aislado es suficiente para descartar *neglect*, por lo que la práctica profesional actual tiende a combinar más de una herramienta.[5]

Dentro de las tareas de cancelación, *Bells Test* y *Letter Cancellation Task* son dos de los paradigmas más representativos. *Bells Test* fue diseñado como una "*cancellation task*" que permite una evaluación cuantitativa y cualitativa del *neglect* visual, ya que el paciente debe identificar estímulos objetivo distribuidos entre distractores y el examinador puede analizar no sólo cuántos elementos encontró, sino también dónde los encontró y cómo exploró la hoja. En términos clínicos, este tipo de tarea se mide mediante el número total de aciertos, las omisiones en hemiespacio izquierdo y derecho, la distribución espacial de los errores y, en algunos protocolos, el orden u organización de

[4] Recorte del campo visual izquierdo: ¿Es heminegligencia o hemianopsia?
[5] Rapid screening for neglect following stroke: A systematic search and European Academy of Neurology recommendations

la búsqueda[6] [7]. Estas características son especialmente relevantes, porque comparten la lógica de una búsqueda visual con “exploración de faro".

La bisección de líneas constituye otro método clásico. En esta prueba, el paciente debe marcar el punto medio de una línea horizontal, y la medida principal es la desviación respecto del centro real, expresada en milímetros o porcentaje de la longitud de la línea. La literatura la describe como una prueba simple y ampliamente usada para el estudio del *neglect*, donde una desviación marcada hacia la derecha en pacientes con lesión hemisférica derecha sugiere *neglect* izquierdo. Aunque sigue siendo una prueba importante por su simplicidad, hoy se considera más útil cuando se la interpreta junto con tareas de cancelación y otras pruebas complementarias.[8] [9]

Además de los tests aislados, los profesionales utilizan baterías estandarizadas como el *Behavioural Inattention Test* (BIT). Esta batería combina varios subtests —cancelación, bisección, copiado, lectura y otros ítems conductuales— y permite obtener tanto un puntaje total como subpuntajes por tarea, comparables con referencias normativas. Hartman-Maeir y Katz concluyeron que los subtests conductuales del BIT presentan validez de constructo y validez predictiva como medidas funcionales del *neglect* unilateral, por lo que el BIT se consolidó como una herramienta clínica útil para vincular el rendimiento neuropsicológico con el funcionamiento cotidiano.[10]

En paralelo, para captar el impacto del *neglect* en actividades reales, se utiliza la *Catherine Bergego Scale* (CBS). Esta escala funcional es especialmente importante porque no se limita al rendimiento en papel, sino que evalúa el *neglect* en situaciones de la vida diaria mediante 10 ítems, cada uno puntuado en una escala de 0 a 3, con un puntaje total de 0 a 30. Azouvi y colaboradores estudiaron sus propiedades

---

[6] The Bells Test: A Qua ntitative and Qua lita tive Test For Visual Neglect
[7] A simple measure of neglect severity - PubMed
[8] Weight and see: Line bisection in neglect reliably measures the allocation of attention, but not the perception of length
[9] Rapid screening for neglect following stroke: A systematic search and European Academy of Neurology recommendations
[10] Validity of the Behavioral Inattention Test (BIT): relationships with functional tasks

psicométricas y compararon su sensibilidad con la de tres tests convencionales, mostrando que la CBS es una referencia clave para valorar la severidad funcional del *neglect*. Por eso, en el estado del arte actual, suele considerarse el complemento funcional de las pruebas neuropsicológicas clásicas.[11]

Recientemente, las tendencias actuales han avanzado hacia la evaluación computarizada. La revisión sistemática de Giannakou señala que los tests computarizados permiten registrar variables continuas y objetivas que no siempre pueden obtenerse con la misma precisión en papel: tiempo de reacción, aciertos por lado, omisiones izquierda/derecha, tiempo total de búsqueda, trayectoria de exploración e índices de sesgo espacial. En otras palabras, mientras que las pruebas tradicionales indican si hubo o no *neglect* y ofrecen una severidad aproximada, los métodos digitales permiten describir con mucho más detalle cómo explora el paciente el espacio. [12]

Finalmente, como línea más avanzada, aparecen los métodos de *eye-tracking* o video-oculografía. En este caso no se mide el movimiento del cursor ni la respuesta final, sino directamente la posición de la mirada, las fijaciones, las sacadas y la latencia de exploración hacia un hemiespacio. Kaufmann y colaboradores mostraron que la video-oculografía durante exploración visual libre tuvo alta sensibilidad y especificidad para detectar *neglect* post-ACV y resultó más sensible que varias pruebas neuropsicológicas convencionales. Aunque no es el método más extendido en la clínica general, sí representa una referencia importante del estado del arte porque muestra hacia dónde evolucionan los sistemas de evaluación objetiva del *neglect*. [13]

---

[11] Behavioral assessment of unilateral neglect: study of the psychometric properties of the Catherine Bergego Scale
[12] Computer-based assessment of unilateral spatial neglect: A systematic review - PubMed
[13] Eyetracking during free visual exploration detects neglect more reliably than paper-pencil tests - PubMed

1.3 Definición de métricas: valores “normales” y “patológicos”

Para definir métricas de un módulo digital de *neglect*, lo más sólido es partir de variables ya validadas en la literatura clínica. En las tareas de cancelación, los profesionales consideran normales los desempeños con baja cantidad de omisiones, distribución aproximadamente simétrica entre ambos lados del espacio y ausencia de sesgo espacial marcado. En cambio, se interpreta como patológico un patrón con más omisiones en el hemiespacio contralesional —habitualmente el izquierdo en lesiones derechas— y con clara asimetría entre izquierda y derecha. El trabajo de Rorden y Karnath es especialmente útil porque propone una medida cuantitativa robusta, el *Center of Cancellation* (CoC), que resume el sesgo espacial de la exploración. En ese estudio, un CoC > 0,081 en el *Bells Test* o > 0,083 en la *Letter Cancellation Task* indicó *neglect* en pacientes con lesión aguda del hemisferio derecho; un valor cercano a 0 corresponde a una exploración más simétrica.

Ese mismo trabajo también aporta valores descriptivos para otras variables, aunque con una salvedad importante: no son normas universales de población sana, sino valores observados en una muestra clínica de pacientes con lesión hemisférica derecha, con y sin *neglect*. En la *Letter Cancellation Task*, los pacientes sin *neglect* mostraron en promedio aproximadamente 28,7 aciertos a la izquierda y 29,2 a la derecha, mientras que los pacientes con *neglect* tuvieron valores mucho menores, por ejemplo alrededor de 4,4 a la izquierda y 18 a la derecha. En el *Bells* Test, los pacientes sin *neglect* presentaron cerca de 13,9 aciertos a la izquierda y 14,2 a la derecha, mientras que en *neglect* los valores fueron menores, como 2,0 a la izquierda y 8,8 a la derecha. En esa lógica, los criterios prácticos de patología fueron más de 4 omisiones contralesionales en la tarea de letras y más de 5 en el *Bells Test*.

En las tareas de copiado, la variable principal suele ser el porcentaje de elementos correctamente reproducidos o, equivalentemente, el porcentaje de omisiones espaciales.

En el mismo estudio de Rorden y Karnath, los pacientes sin *neglect* obtuvieron en la tarea de copiado un rendimiento cercano al 96% correcto, mientras que los pacientes con *neglect* mostraron un valor mucho menor, alrededor de 47%, y se consideró patológico un nivel de omisiones superior al 12,5%. Esto muestra que, aún fuera de la búsqueda visual explícita, el *neglect* puede traducirse en una pérdida cuantificable de componentes del lado contralesional. [14]

Para el caso de la CBS, la métrica es distinta porque no mide tiempos ni aciertos visuales, sino severidad funcional. El puntaje total va de 0 a 30, con valores más altos indicando mayor impacto del *neglect* sobre las actividades cotidianas. En este caso no se habla tanto de "normal" versus "patológico" con un único corte universal, sino de un continuo de severidad: 0 corresponde a ausencia de *neglect* funcional observable, mientras que puntajes crecientes reflejan limitaciones cada vez más importantes en la vida diaria. [15] [16]

En el caso de la evaluación computarizada, las métricas más pertinentes son el tiempo de reacción, la cantidad de objetivos detectados a izquierda y derecha, el porcentaje de omisiones contralesionales, el tiempo total de búsqueda, la trayectoria de exploración y algún índice de sesgo espacial análogo al CoC. La revisión de Giannakou respalda precisamente estas variables como las más usadas en sistemas digitales. Sin embargo, esa revisión no establece un único valor universal de "normalidad" en milisegundos para tiempo de reacción, porque los resultados dependen del diseño de la tarea, la interfaz, el tamaño de la pantalla, el tipo de estímulo y la población estudiada. [17]

---

[14] A simple measure of neglect severity - PubMed
[15] Behavioral assessment of unilateral neglect: study of the psychometric properties of the Catherine Bergego Scale - PubMed
[16] Conventional and Functional Assessment of Spatial Neglect: Clinical Practice Suggestions - PMC
[17] Computer-based assessment of unilateral spatial neglect: A systematic review - PubMed

2. Desarrollo de la herramienta

El test de **exploración de faro** es una herramienta digital orientada a evaluar la **exploración visuoespacial** y detectar signos de **heminegligencia**, especialmente hacia el hemiespacio izquierdo. La tarea se desarrolla en una pantalla oscura donde el cursor funciona como una "linterna" que permite visualizar sólo una región limitada del espacio. El paciente debe explorar activamente la pantalla y hacer clic sobre los objetos ocultos.

En cada nivel se presentan **12 objetos en total**, distribuidos de manera no uniforme: 8 en el lado izquierdo y 4 en el derecho, lo cual permite evaluar de forma sensible posibles sesgos atencionales. El test cuenta con un **límite de tiempo** de 90 segundos por nivel, pudiendo finalizar antes si el paciente logra encontrar todos los objetos. Durante la tarea se registran múltiples variables:

- cantidad de objetos encontrados y omitidos,
- distribución de aciertos entre ambos hemiespacios (izquierdo y derecho),
- tiempos de detección,
- tiempo total de ejecución,
- errores de selección (clics en zonas sin objetos),
- y la trayectoria del cursor, que permite reconstruir el patrón de exploración visual.

El rendimiento se sintetiza mediante un **puntaje** por nivel en **escala de 0 a 100**, que combina tres componentes:

$$Puntaje\ del\ nivel = puntaje\ por\ aciertos + puntaje\ por\ velocidad - penalización\ por\ errores$$

Donde:

- $puntaje\ por\ aciertos = (objetos_encontrados\ /\ total_objetos) * 70$
- $puntaje\ por\ velocidad = (1 - tiempo_utilizado\ /\ tiempo_máximo) * 30$
- $penalización\ por\ errores = errores * 2$

En primer lugar, un puntaje por **aciertos**, con un máximo de 70 puntos, proporcional a la cantidad de objetos encontrados sobre el total. En segundo lugar, un puntaje por **velocidad**, con un máximo de 30 puntos, que depende del tiempo total utilizado en relación con el tiempo máximo disponible (a menor tiempo, mayor puntaje). Finalmente, se aplica una **penalización por errores**, restando 2 puntos por cada clic incorrecto. De esta manera, el puntaje refleja tanto la efectividad como la eficiencia y la precisión de la exploración.

El test se organiza en **tres niveles de dificultad creciente**, en los cuales se modifican parámetros visuales que afectan la demanda perceptual. A medida que aumenta el nivel, disminuye el radio de la linterna, reduciendo el campo visual disponible, y también disminuye el contraste entre los objetos y el fondo, lo que dificulta su detección. Estas modificaciones incrementan la exigencia del sistema atencional y permiten evaluar el desempeño del paciente en condiciones progresivamente más complejas.

En conjunto, el test permite analizar no solo el rendimiento global, sino también el patrón espacial de la exploración, siendo especialmente relevante la comparación entre estímulos detectados en el lado izquierdo y derecho. La integración de estas métricas, junto con el registro estructurado de los datos por paciente y sesión, posibilita un análisis objetivo y el seguimiento longitudinal del desempeño, facilitando la evaluación de la evolución clínica en contextos de rehabilitación neuropsicológica.

3. Análisis de un caso con desempeño dentro de valores normales

A partir de los datos obtenidos en una ejecución del test en sus 3 niveles de complejidad, se realizó un análisis comparativo con los valores reportados en la literatura para tareas de exploración visuoespacial.

Para nivel 1, el resultado de dicho test arrojó los siguientes parámetros:

- Objetos encontrados: 12 (100%)
- Omisiones: 0
- Aciertos izquierda: 8 / 8
- Aciertos derecha: 4 / 4
- Tiempo total: ~18,2 s
- Errores: 2
- Puntaje total: 89.94/100

En primer lugar, la ausencia total de omisiones y la detección completa de estímulos en ambos hemiespacios constituyen un indicador fuerte de desempeño dentro de la normalidad. En la bibliografía, los pacientes sin neglect muestran precisamente este patrón: alta tasa de aciertos y exploración completa del campo visual.

En segundo lugar, se observa una distribución proporcional de aciertos izquierda–derecha, teniendo en cuenta que había más estímulos en el lado izquierdo (8 vs 4). Esto es clave, porque en pacientes con neglect izquierdo se esperaría una reducción marcada de aciertos en ese lado, lo cual no se evidencia en este caso.

Si se interpreta este resultado en términos del índice de sesgo espacial propuesto por Rorden y Karnath (Center of Cancellation, CoC), el patrón observado sería compatible con un valor cercano a 0, es decir, exploración simétrica, dentro del rango considerado normal (CoC ≈ 0). Dicho parámetro se calcula como el promedio ponderado de los estímulos omitidos o detectados

Con respecto al análisis temporal y estrategia de exploración, el tiempo total de búsqueda de aproximadamente 18 s, se encuentra dentro de un rango esperable para una tarea de baja complejidad, y no se observa abandono temprano ni finalización por timeout.

Analizando la secuencia de detección, inicialmente se explora el lado izquierdo, lo cual es esperable por la incorporación del sistema de flechas de ayuda, haciendo que se complete y luego pasando al lado derecho-

Se registraron 2 errores, correspondientes a click sin objetivo,lo cual es bajo y esperable en personas sin alteraciones, con lo cual no son indicativos de neglects por no estar acompañados de omisiones sistemáticas o sesgo espacial.

A nivel de comparar el desempeño de los 3 niveles se objetivo lo siguiente:

- Nivel 1 (Fácil): Con un radio de linterna de 150 px, el usuario encontró los 12 objetos en solo 18.17 segundos, obteniendo un puntaje de 89.94. Este nivel sirve como línea de base para evaluar la comprensión de la tarea.
- Nivel 2 (Medio): Al reducir el radio a 100 px, el tiempo total de resolución aumentó a 34.88 segundos. A pesar de la mayor exigencia visual, el usuario mantuvo una precisión perfecta (12/12 objetos).
- Nivel 3 (Difícil): Con un radio de 90 px y cambios en los niveles de contraste, se observa el mayor impacto. El usuario no logró completar la tarea en los 90 segundos límite, omitiendo un objeto en el lado izquierdo, lo cual indica que hay un aumento de la complejidad de la tarea.

Esta progresión es útil para el caso de tratamiento de pacientes con neglect, ya que permite al terapeuta ajustar el "desafío justo": empezar con radios amplios para fomentar el éxito inicial y reducirlo gradualmente para entrenar la atención sostenida y el rastreo sistemático.

3.1 Evaluación de métricas y comparación con bibliografía:

Ahora bien, para evaluar la validez del test desarrollado, se establecieron valores de referencia proporcionales basados en tests de cancelación estandarizados (como el Bells Test y el Letter Cancellation Task). Dado que la bibliografía clínica establece que la omisión de más del 15-20% de los estímulos en un hemiespacio es indicativa de heminegligencia, hemos transpuesto estos umbrales a nuestra muestra de 12 objetos. Así, mientras que una persona sana alcanza el 100% de aciertos en niveles de baja

complejidad, un resultado patológico en nuestra herramienta se definiría por la omisión sistemática de estímulos en el lado izquierdo, independientemente del tiempo empleado.

A continuación, se presentan las métricas definidas para la evaluación del desempeño en la tarea de exploración visuoespacial, junto con sus valores de referencia e interpretación clínica. Los parámetros incluidos combinan tanto indicadores cuantitativos como cualitativos.

En particular, la tasa de aciertos, la simetría espacial y el índice de Center of Cancellation (CoC) se basan en criterios bibliográficos como el Bells Test (Gauthier et al.) y el CoC propuesto por Rorden y Karnath, adaptados proporcionalmente al diseño del test desarrollado. Si bien estos umbrales no provienen del análisis de una cohorte propia, constituyen referencias válidas para una primera aproximación a la detección de patrones compatibles con heminegligencia.

Por otro lado, se incorporan métricas de carácter cualitativo, como la resistencia a la complejidad y la trayectoria de exploración, que permiten complementar el análisis a partir de la estrategia de búsqueda del usuario. Estas variables, aunque no se expresan mediante umbrales estrictos, resultan relevantes en la práctica clínica para la interpretación del comportamiento exploratorio.

Variable de Medida	Sujeto sano evaluado	Umbral Patológico	Interpretación Clínica
Tasa de Aciertos *(Aciertos≠Puntaje)*	100% (12/12) en Niveles 1 y 2.	< 88,5% sugiere ***déficit atencional general***	La omisión de estímulos sugiere falla en el rastreo visual. Ref: Bells Test (Gauthier et al.)
Simetría Espacial	Encuentra 8/8 a la Izq.	**Sesgo > 2**	El neglect se manifiesta

	y 4/4 a la Der.	**omisiones en lado Izq.** (17% - 20%)	como un gradiente de derecha a izquierda. Ref: Bells Test (Gauthier et al.)
Índice CoC (Center of Cancellation)	Próximo a 0.0 (Simétrico).	**> 0.081**	Valor estándar para confirmar sesgo espacial (Rorden & Karnath).
Resistencia a la Complejidad	Mantiene precisión con radio de 100px.	**Colapso de aciertos al reducir radio.**	Una persona con neglect depende de un campo visual amplio; la restricción (faro pequeño) agrava el síntoma.
Trayectoria de Exploración	Rastreo sistemático y organizado (ej. de izquierda a derecha o en "S").	**Desordenada, confinada al lado derecho o con "saltos" erráticos.**	El archivo JSON registra coordenadas cada 20ms, permitiendo reconstruir el recorrido exacto.

Tabla 1: Análisis de resultados del test en sujeto sano y definición de umbrales patológicos según análisis bibliográfico.

Anexo I: Código final

```
import tkinter as tk
import random
import math
import json
import os
from datetime import datetime
import time


# ============================================================
# CONFIGURACIÓN GENERAL
# ============================================================

WINDOW_WIDTH = 1200
WINDOW_HEIGHT = 700

BACKGROUND_COLOR = "#050505"
TEXT_COLOR = "white"
OBJECT_COLOR = "#b0b0b0"
FOUND_OBJECT_COLOR = "#50d070"
FLASHLIGHT_OUTLINE_COLOR = "#fff7b2"

FLASHLIGHT_RADIUS = 120
OBJECT_RADIUS = 18

TOTAL_OBJECTS = 12
LEFT_SIDE_OBJECTS = 8
RIGHT_SIDE_OBJECTS = TOTAL_OBJECTS - LEFT_SIDE_OBJECTS

MIN_DISTANCE_BETWEEN_OBJECTS = 80
MAX_TEST_DURATION_SEC = 90

RESULTS_FOLDER = "results"
TRAJECTORY_SAMPLING_MS = 20

# Puntaje
MAX_ACCURACY_SCORE = 70
MAX_SPEED_SCORE = 30
ERROR_PENALTY = 2
```

```python
# ============================================================
# FUNCIONES AUXILIARES
# ============================================================

def ensure_results_folder():
    if not os.path.exists(RESULTS_FOLDER):
        os.makedirs(RESULTS_FOLDER)


def distance(p1, p2):
    return math.hypot(p1[0] - p2[0], p1[1] - p2[1])


def sanitize_filename(text):
    safe = "".join(c for c in text if c.isalnum() or c in ("_", "-"))
    return safe.strip() if safe.strip() else "sin_nombre"


def save_results_to_json(data):
    ensure_results_folder()
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    test_info = data.get("test_info", {})
    level = test_info.get("level", 1)
    patient_name = test_info.get("patient_name", "sin_nombre")
    session_number = test_info.get("session_number", "sin_sesion")

    safe_name = sanitize_filename(patient_name)

    filename = os.path.join(
        RESULTS_FOLDER,

f"exploracion_faro_{safe_name}_sesion{session_number}_nivel{level}_{tim
estamp}.json"
    )

    with open(filename, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=4, ensure_ascii=False)

    return filename


def generate_non_overlapping_positions():
```

```
    positions = []

    def valid_position(new_pos):
        for pos in positions:
            if distance(pos, new_pos) < MIN_DISTANCE_BETWEEN_OBJECTS:
                return False
        return True

    left_count = 0
    attempts = 0
    while left_count < LEFT_SIDE_OBJECTS and attempts < 5000:
        x = random.randint(60, WINDOW_WIDTH // 2 - 80)
        y = random.randint(100, WINDOW_HEIGHT - 80)
        candidate = (x, y)
        if valid_position(candidate):
            positions.append(candidate)
            left_count += 1
        attempts += 1

    right_count = 0
    attempts = 0
    while right_count < RIGHT_SIDE_OBJECTS and attempts < 5000:
        x = random.randint(WINDOW_WIDTH // 2 + 80, WINDOW_WIDTH - 60)
        y = random.randint(100, WINDOW_HEIGHT - 80)
        candidate = (x, y)
        if valid_position(candidate):
            positions.append(candidate)
            right_count += 1
        attempts += 1

    random.shuffle(positions)
    return positions


def calculate_level_score(objects_found, total_objects, total_time_ms,
max_time_sec, error_count):
    accuracy_score = (objects_found / total_objects) *
MAX_ACCURACY_SCORE

    time_ratio = min(total_time_ms / (max_time_sec * 1000), 1.0)
    speed_score = (1 - time_ratio) * MAX_SPEED_SCORE

    error_penalty = error_count * ERROR_PENALTY
```

```
    raw_score = accuracy_score + speed_score - error_penalty
    final_score = max(0, min(100, round(raw_score, 2)))

    return {
        "accuracy_score": round(accuracy_score, 2),
        "speed_score": round(speed_score, 2),
        "error_penalty": round(error_penalty, 2),
        "level_score": final_score
    }


# ============================================================
# CLASE DEL OBJETO OCULTO
# ============================================================

class HiddenObject:
    def __init__(self, obj_id, position):
        self.id = obj_id
        self.x, self.y = position
        self.radius = OBJECT_RADIUS
        self.found = False
        self.found_time_ms = None
        self.click_position = None
        self.side = "left" if self.x < WINDOW_WIDTH / 2 else "right"

    def is_clicked(self, mouse_pos):
        return distance((self.x, self.y), mouse_pos) <= self.radius


# ============================================================
# APLICACIÓN PRINCIPAL
# ============================================================

class ExploracionFaroApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Exploración de Faro - Neglect")
        self.root.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.root.configure(bg=BACKGROUND_COLOR)
        self.root.resizable(False, False)

        self.mouse_x = WINDOW_WIDTH // 2
```

```
self.mouse_y = WINDOW_HEIGHT // 2

self.level = 1
self.bg_color = BACKGROUND_COLOR
self.obj_color = OBJECT_COLOR
self.found_color = FOUND_OBJECT_COLOR
self.flashlight_radius = FLASHLIGHT_RADIUS

self.test_running = False
self.test_finished = False
self.test_start_time = None
self.end_reason = None

self.objects = []
self.cursor_trajectory = []
self.click_events = []
self.error_count = 0
self.last_error_timestamp = None
self.json_path = None
self.last_level_score_data = None

# Datos del paciente
self.patient_name = ""
self.session_number = ""
self.active_field = "name"
self.input_completed = False

self.canvas = tk.Canvas(
    self.root,
    width=WINDOW_WIDTH,
    height=WINDOW_HEIGHT,
    bg=BACKGROUND_COLOR,
    highlightthickness=0
)
self.canvas.pack()

self.root.bind("<Motion>", self.on_mouse_move)
self.root.bind("<Button-1>", self.on_left_click)
self.root.bind("<Escape>", self.on_escape)
self.root.bind("<space>", self.on_space)

self.root.bind("<Key>", self.on_key_press)
self.root.bind("<Tab>", self.on_tab_press)
```

```
        self.root.bind("<Return>", self.on_enter_press)

        self.draw_instructions()

    # ==========================================================
    # PANTALLA INICIAL
    # ==========================================================

    def draw_instructions(self, error_message=""):
        self.canvas.delete("all")

        self.canvas.create_text(
            WINDOW_WIDTH // 2, 70,
            text="Exploración de Faro",
            fill=TEXT_COLOR,
            font=("Arial", 28, "bold")
        )

        self.canvas.create_text(
            WINDOW_WIDTH // 2, 120,
            text="Ingresar datos antes de comenzar",
            fill="#b4dcff",
            font=("Arial", 18, "bold")
        )

        self.canvas.create_text(
            WINDOW_WIDTH // 2, 190,
            text="Nombre del paciente:",
            fill=TEXT_COLOR,
            font=("Arial", 18)
        )

        name_color = "#00aaff" if self.active_field == "name" else
"#cccccc"
        self.canvas.create_rectangle(350, 215, 850, 260,
outline=name_color, width=2)
        self.canvas.create_text(
            370, 238,
            anchor="w",
            text=self.patient_name if self.patient_name else "Escriba
aquí",
            fill=TEXT_COLOR if self.patient_name else "#777777",
            font=("Arial", 18)
```

```
        )

        self.canvas.create_text(
            WINDOW_WIDTH // 2, 320,
            text="Número de sesión:",
            fill=TEXT_COLOR,
            font=("Arial", 18)
        )

        session_color = "#00aaff" if self.active_field == "session"
else "#cccccc"
        self.canvas.create_rectangle(480, 345, 720, 390,
outline=session_color, width=2)
        self.canvas.create_text(
            500, 368,
            anchor="w",
            text=self.session_number if self.session_number else "Ej:
1",
            fill=TEXT_COLOR if self.session_number else "#777777",
            font=("Arial", 18)
        )

        if not self.input_completed:
            self.canvas.create_text(
                WINDOW_WIDTH // 2, 450,
                text="TAB para cambiar de campo",
                fill="#cccccc",
                font=("Arial", 15)
            )

            self.canvas.create_text(
                WINDOW_WIDTH // 2, 480,
                text="Backspace para borrar",
                fill="#cccccc",
                font=("Arial", 15)
            )

            self.canvas.create_text(
                WINDOW_WIDTH // 2, 510,
                text="ENTER para INICIAR",
                fill="#00cc66",
                font=("Arial", 16, "bold")
            )
```

```
        start_text = "Completá los datos y presioná ENTER para iniciar"
        if self.input_completed:
            start_text = "Datos listos. Ahora presioná 1 (Fácil), 2
(Medio) o 3 (Difícil)"

        self.canvas.create_text(
            WINDOW_WIDTH // 2, 550,
            text=start_text,
            fill="#b4dcff",
            font=("Arial", 18, "bold")
        )

        self.canvas.create_text(
            WINDOW_WIDTH // 2, 595,
            text="ESC para cerrar",
            fill="#c0c0c0",
            font=("Arial", 16)
        )

        if error_message:
            color = "red"

            if "correctamente" in error_message.lower():
                color = "#00cc66"  # verde

            self.canvas.create_text(
                WINDOW_WIDTH // 2, 640,
                text=error_message,
                fill=color,
                font=("Arial", 16, "bold")
            )

    # ==========================================================
    # ENTRADA DE DATOS
    # ==========================================================

    def on_tab_press(self, event=None):
        if self.test_running:
            return "break"

        if self.active_field == "name":
            self.active_field = "session"
```

```
        else:
            self.active_field = "name"

        self.draw_instructions()
        return "break"

    def on_enter_press(self, event=None):
        if self.test_running:
            return "break"

        if self.patient_name.strip() and self.session_number.strip():
            self.input_completed = True
            self.draw_instructions("Datos cargados correctamente")
        else:
            self.input_completed = False
            self.draw_instructions("Completá nombre y número de sesión
antes de confirmar.")

        return "break"


    def on_key_press(self, event):
        if self.test_running:
            return

        if event.keysym in ["Tab", "Return", "Escape"]:
            return

        if event.keysym == "BackSpace":
            self.input_completed = False
            if self.active_field == "name":
                self.patient_name = self.patient_name[:-1]
            else:
                self.session_number = self.session_number[:-1]
            self.draw_instructions()
            return

        char = event.char
        if not char:
            return

        # Si los datos ya fueron confirmados, 1/2/3 sirven para elegir
nivel
```

```
        if self.input_completed and char in ["1", "2", "3"]:
            self.start_test(level=int(char))
            return

        # Si todavía no confirmó, se usan para escribir
        if self.active_field == "name":
            if char.isprintable():
                self.input_completed = False
                self.patient_name += char
        elif self.active_field == "session":
            if char.isdigit():
                self.input_completed = False
                self.session_number += char

        self.draw_instructions()
    # ==========================================================
    # INICIO DEL TEST
    # ==========================================================

    def start_test(self, event=None, level=1):
        if self.test_running:
            return

        if not self.input_completed:
            self.draw_instructions("Primero cargá nombre y número de
sesión y presioná ENTER.")
            return

        if not self.patient_name.strip() or not
self.session_number.strip():
            self.draw_instructions("Completá nombre y número de sesión
antes de comenzar")
            return

        self.level = level
        if level == 1:
            self.bg_color = "#050505"
            self.obj_color = "#ffffff"
            self.found_color = "#50d070"
            self.flashlight_radius = 150
        elif level == 2:
            self.bg_color = "#151515"
            self.obj_color = "#808080"
```

```
            self.found_color = "#40a050"
            self.flashlight_radius = 100
        else:
            self.bg_color = "#2a2a2a"
            self.obj_color = "#353535"
            self.found_color = "#306040"
            self.flashlight_radius = 70

        self.canvas.configure(bg=self.bg_color)

        positions = generate_non_overlapping_positions()
        self.objects = [HiddenObject(i + 1, pos) for i, pos in
enumerate(positions)]

        self.cursor_trajectory = []
        self.click_events = []
        self.error_count = 0
        self.last_error_timestamp = None
        self.json_path = None
        self.last_level_score_data = None

        self.test_running = True
        self.test_finished = False
        self.test_start_time = time.time()
        self.end_reason = None

        self.update_test()
        self.record_trajectory()

    # ==========================================================
    # EVENTOS
    # ==========================================================

    def on_mouse_move(self, event):
        self.mouse_x = event.x
        self.mouse_y = event.y
        if self.test_running:
            self.draw_test_scene()

    def on_left_click(self, event):
        if not self.test_running:
            return
```

```
        elapsed_ms = self.get_elapsed_ms()
        click_x, click_y = event.x, event.y
        hit = False

        for obj in self.objects:
            if not obj.found and obj.is_clicked((click_x, click_y)):
                obj.found = True
                obj.found_time_ms = elapsed_ms
                obj.click_position = {"x": click_x, "y": click_y}
                hit = True

                self.click_events.append({
                    "time_ms": elapsed_ms,
                    "x": click_x,
                    "y": click_y,
                    "type": "hit",
                    "object_id": obj.id,
                    "object_side": obj.side
                })
                break

        if not hit:
            self.error_count += 1
            self.last_error_timestamp = time.time()

            self.click_events.append({
                "time_ms": elapsed_ms,
                "x": click_x,
                "y": click_y,
                "type": "error",
                "object_id": None,
                "object_side": None
            })

        self.draw_test_scene()

        if all(obj.found for obj in self.objects):
            self.finish_test("all_objects_found")

    def on_escape(self, event=None):
        if self.test_running:
            self.finish_test("user_cancelled")
        self.root.after(100, self.root.destroy)
```

```
    def on_space(self, event=None):
        if self.test_finished and self.level < 3 and self.end_reason !=
"user_cancelled":
            self.start_test(level=self.level + 1)

    # ==========================================================
    # TIEMPO
    # ==========================================================

    def get_elapsed_ms(self):
        if not self.test_start_time:
            return 0
        return int((time.time() - self.test_start_time) * 1000)

    def get_elapsed_sec(self):
        return self.get_elapsed_ms() / 1000.0

    # ==========================================================
    # REGISTRO DE TRAYECTORIA
    # ==========================================================

    def record_trajectory(self):
        if not self.test_running:
            return

        self.cursor_trajectory.append({
            "time_ms": self.get_elapsed_ms(),
            "x": self.mouse_x,
            "y": self.mouse_y
        })

        self.root.after(TRAJECTORY_SAMPLING_MS, self.record_trajectory)

    # ==========================================================
    # ACTUALIZACIÓN GENERAL
    # ==========================================================

    def update_test(self):
        if not self.test_running:
            return

        if self.get_elapsed_sec() >= MAX_TEST_DURATION_SEC:
```

```
            self.finish_test("time_limit")
            return

        self.draw_test_scene()
        self.root.after(16, self.update_test)

    # ==========================================================
    # DIBUJO DEL TEST
    # ==========================================================

    def draw_test_scene(self):
        self.canvas.delete("all")

        self.canvas.create_rectangle(
            0, 0, WINDOW_WIDTH, WINDOW_HEIGHT,
            fill=self.bg_color, outline=self.bg_color
        )

        for obj in self.objects:
            d = distance((obj.x, obj.y), (self.mouse_x, self.mouse_y))
            if d <= self.flashlight_radius:
                color = self.found_color if obj.found else
self.obj_color
                self.canvas.create_oval(
                    obj.x - obj.radius, obj.y - obj.radius,
                    obj.x + obj.radius, obj.y + obj.radius,
                    fill=color, outline=color
                )

        self.canvas.create_oval(
            self.mouse_x - self.flashlight_radius, self.mouse_y -
self.flashlight_radius,
            self.mouse_x + self.flashlight_radius, self.mouse_y +
self.flashlight_radius,
            outline=FLASHLIGHT_OUTLINE_COLOR, width=2
        )

        self.canvas.create_oval(
            self.mouse_x - 4, self.mouse_y - 4,
            self.mouse_x + 4, self.mouse_y + 4,
            fill="#fff7c0", outline="#fff7c0"
        )
```

```
        if self.level == 1:
            left_unfound = any(not obj.found for obj in self.objects if
obj.side == "left")
            if left_unfound:
                self.canvas.create_line(5, 0, 5, WINDOW_HEIGHT,
fill="red", width=10)
                offset = int(time.time() * 50) % 30
                for obj in self.objects:
                    if not obj.found and obj.side == "left":
                        start_x = WINDOW_WIDTH // 2 + 60 - offset
                        end_x = WINDOW_WIDTH // 2 - 60 - offset
                        self.canvas.create_line(
                            start_x, obj.y, end_x, obj.y,
                            arrow=tk.LAST, fill="#ffdd00", width=4
                        )

        if self.last_error_timestamp is not None:
            if time.time() - self.last_error_timestamp < 0.25:
                self.canvas.create_oval(
                    self.mouse_x - 20, self.mouse_y - 20,
                    self.mouse_x + 20, self.mouse_y + 20,
                    outline="red", width=3
                )

        found_count = sum(obj.found for obj in self.objects)
        time_left = max(0, int(MAX_TEST_DURATION_SEC -
self.get_elapsed_sec()))
        info_text = (
            f"Paciente: {self.patient_name}    "
            f"Número de sesión: {self.session_number}    "
            f"Nivel: {self.level}    "
            f"Encontrados: {found_count}/{TOTAL_OBJECTS}    "
            f"Tiempo restante: {time_left}s    "
            f"Errores: {self.error_count}"
        )

        timer_color = TEXT_COLOR
        if 0 < time_left <= 10:
            if int(time.time() * 4) % 2 == 0:
                timer_color = "red"
                self.canvas.create_text(
                    WINDOW_WIDTH // 2, WINDOW_HEIGHT // 2 - 100,
                    text=f"¡QUEDAN {time_left} SEGUNDOS!",
```

```
                    fill="red",
                    font=("Arial", 36, "bold")
                )

        self.canvas.create_text(
            20, 20,
            anchor="nw",
            text=info_text,
            fill=timer_color,
            font=("Arial", 14, "bold")
        )

    # ==========================================================
    # FINALIZACIÓN
    # ==========================================================

    def finish_test(self, reason):
        if not self.test_running:
            return

        self.test_running = False
        self.test_finished = True
        self.end_reason = reason

        total_time_ms = self.get_elapsed_ms()
        total_found = sum(obj.found for obj in self.objects)
        left_found = sum(1 for obj in self.objects if obj.side ==
"left" and obj.found)
        right_found = sum(1 for obj in self.objects if obj.side ==
"right" and obj.found)

        score_data = calculate_level_score(
            objects_found=total_found,
            total_objects=TOTAL_OBJECTS,
            total_time_ms=total_time_ms,
            max_time_sec=MAX_TEST_DURATION_SEC,
            error_count=self.error_count
        )
        self.last_level_score_data = score_data

        result_data = {
            "test_info": {
                "test_name": "Exploración de Faro",
```

```
        "test_type": "Neglect",
        "level": self.level,
        "patient_name": self.patient_name,
        "session_number": self.session_number,
        "date_time": datetime.now().isoformat(),
        "window_width": WINDOW_WIDTH,
        "window_height": WINDOW_HEIGHT,
        "max_duration_sec": MAX_TEST_DURATION_SEC,
        "flashlight_radius_px": self.flashlight_radius,
        "total_objects": TOTAL_OBJECTS,
        "left_side_objects": LEFT_SIDE_OBJECTS,
        "right_side_objects": RIGHT_SIDE_OBJECTS,
        "trajectory_sampling_ms": TRAJECTORY_SAMPLING_MS
    },
    "performance": {
        "total_time_ms": total_time_ms,
        "objects_found": total_found,
        "objects_missed": TOTAL_OBJECTS - total_found,
        "left_found": left_found,
        "right_found": right_found,
        "error_count": self.error_count,
        "end_reason": self.end_reason
    },
    "score": {
        "accuracy_score": score_data["accuracy_score"],
        "speed_score": score_data["speed_score"],
        "error_penalty": score_data["error_penalty"],
        "level_score": score_data["level_score"],
        "score_scale": "0_to_100"
    },
    "objects": [
        {
            "id": obj.id,
            "x": obj.x,
            "y": obj.y,
            "side": obj.side,
            "found": obj.found,
            "found_time_ms": obj.found_time_ms,
            "click_position": obj.click_position
        }
        for obj in self.objects
    ],
    "click_events": self.click_events,
```

```
            "cursor_trajectory": self.cursor_trajectory
        }

        self.json_path = save_results_to_json(result_data)
        self.draw_end_screen(result_data)

    # ==========================================================
    # PANTALLA FINAL
    # ==========================================================

    def draw_end_screen(self, result_data):
        self.canvas.delete("all")

        perf = result_data["performance"]
        score = result_data["score"]

        self.canvas.create_text(
            WINDOW_WIDTH // 2, 80,
            text="Test finalizado",
            fill=TEXT_COLOR,
            font=("Arial", 28, "bold")
        )


        y = 145

        self.canvas.create_text(
            WINDOW_WIDTH // 2, y,
            text=f"Paciente: {self.patient_name}   |   Número de
sesión: {self.session_number}",
            fill="#eeeeee",
            font=("Arial", 19, "bold")
        )

        y += 40


        self.canvas.create_text(
            WINDOW_WIDTH // 2, y,
            text=f"NIVEL {self.level}",
            fill="#66ccff",
            font=("Arial", 24, "bold")
        )
```

```
y += 55

self.canvas.create_text(
    WINDOW_WIDTH // 2, y,
    text=f"{score['level_score']}/100",
    fill="#00ff88",
    font=("Arial", 40, "bold")
)

y += 45

self.canvas.create_text(
    WINDOW_WIDTH // 2, y,
    text="Puntaje del nivel",
    fill="#aaaaaa",
    font=("Arial", 18)
)

y += 50

info_lines = [
    f"Aciertos: {perf['objects_found']} de {TOTAL_OBJECTS}",
    f"Lado izquierdo: {perf['left_found']}",
    f"Lado derecho: {perf['right_found']}",
    f"Errores: {perf['error_count']}",
    f"Puntaje por aciertos: {score['accuracy_score']}",
    f"Puntaje por velocidad: {score['speed_score']}",
    f"Penalización por errores: -{score['error_penalty']}",
    f"Archivo guardado en: {self.json_path}"
]

for line in info_lines:
    self.canvas.create_text(
        WINDOW_WIDTH // 2, y,
        text=line,
        fill="#cccccc",
        font=("Arial", 14)
    )
    y += 28

y += 20
```

```
        if self.level < 3 and self.end_reason != "user_cancelled":
            self.canvas.create_text(
                WINDOW_WIDTH // 2, y,
                text=f"Presioná ESPACIO para avanzar al Nivel
{self.level + 1}",
                fill="#00aaff",
                font=("Arial", 20, "bold")
            )
            y += 40

        self.canvas.create_text(
            WINDOW_WIDTH // 2, y,
            text="Presioná ESC para salir",
            fill="#ffaa00",
            font=("Arial", 20, "bold")
        )


# ============================================================
# PUNTO DE ENTRADA
# ============================================================

def main():
    root = tk.Tk()
    app = ExploracionFaroApp(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```

Anexo II: Prompts utilizados con la IA

1. Se le solicitó lo básico en cuanto al desarrollo del programa:

Desarrollar el script de evaluación cumpliendo con:

- Mecánica de Evaluación: El test debe ser claro, simple y accesible (considerar visión y motricidad del paciente).
- Captura de Datos: El software debe registrar cada evento relevante (tiempos de respuesta, errores, trayectorias).
- Exportación Estándar: Al finalizar el test, el programa debe generar un archivo .json en una carpeta llamada /results con la estructura técnica acordada.

Quiero que realices todo esto para Exploración de Faro (Neglect): Pantalla oscura donde el cursor es una linterna. El paciente debe hallar objetos ocultos, principalmente en el margen izquierdo.

Quiero un código en python que se pueda ejecutar en visual studio code. No uses pygame.

2. Se agregaron los niveles:

Agregar al test actual diferentes niveles de complejidad, en principio 3 (fácil, medio y difícil), seleccionables desde la pantalla inicial con teclas 1, 2 y 3.

- Cada nivel debe ajustar:
    - Contraste de los objetos
    - Color de fondo
    - Tamaño del radio de la linterna
- En el nivel fácil:
    - Agregar ayudas visuales en ambos lados (líneas, flechas o guías) para estimular la exploración en pacientes con heminegligencia.
- Agregar una alerta visual cuando queden 10 segundos:
    - Puede ser cambio de color del temporizador o mensaje en pantalla.
- Modificar el guardado de resultados:
    - El archivo JSON debe incluir el nivel
    - El nombre del archivo debe tener el formato: exploracion_faro_nivelX_timestamp.json

3. Se incorporaron mejoras:

Agregar al desarrollo del test las siguientes funcionalidades:

- Definir un puntaje independiente para cada nivel, en escala de 0 a 100. El puntaje debe calcularse en función de:
    - cantidad de objetos encontrados,
    - tiempo empleado,
    - errores cometidos.
- Solicitar antes de comenzar:
    - nombre del paciente,
    - número de sesión.
- Registrar estos datos en el archivo *.json* y utilizarlos también en el nombre del archivo exportado.
- Asegurar que el número de sesión pueda cargarse correctamente sin interferir con la selección del nivel.
- Hacer que la selección de nivel ocurra solo después de confirmar los datos del paciente. De manera que no se confunda la selección del nivel con la escritura del número de sesión.